



Atividades Práticas - TEMA 8

Manipulação de Dados



8 atividades



OneCompiler



Datas |



Callbacks |



CRUD



Atividades

1 Criando e Exibindo uma Data

Enunciado:

Complete o código para criar uma **data atual** e exibir no console.

Use o objeto `Date` para criar a data.

 **Dica:** `new Date()` cria a data atual. Use `console.log()` para exibir.

 atividade-1.js

```
// ATIVIDADE 1: Criando uma data
// Complete o código abaixo

let agora = [ ? ] Date();

console.log(agora);
```

 Cole aqui o **código COMPLETO** que você escreveu (com as lacunas preenchidas):

```
let agora = new Date();

console.log(agora);
```

 Verificar

 Limpar



Perfeito! Código completo e correto!



Todas as lacunas foram preenchidas corretamente!



Código esperado (completo):

```
let agora = new Date();

console.log(agora);
```

2 Criando uma Data Fixa

📌 Enunciado:

Complete o código para criar uma **data fixa** com os valores **15 de maio de 2000**.

Use o objeto `Date` com parâmetros: ano, mês, dia .

Lembre-se: o mês começa em **0** (Janeiro = 0).

💡 **Dica:** `new Date(2000, 4, 15)` cria a data 15/05/2000 (maio = 4).

atividade-2.js

```
// ATIVIDADE 2: Criando uma data fixa
// Complete o código abaixo

let data = new Date([ ? ], [ ? ], [ ? ]);

console.log(data);
```

📋 Cole aqui o **código COMPLETO** que você escreveu (com as lacunas preenchidas):

```
let data = new Date(2000, 4, 15);

console.log(data);
```

✅ Verificar

🔄 Limpar

🎉 **Perfeito! Código completo e correto!**

✅ Todas as lacunas foram preenchidas corretamente!

💡 **Código esperado (completo):**

```
let data = new Date(2000, 4, 15);

console.log(data);
```

3 Acessando Dia, Mês e Ano

Enunciado:

Complete o código para criar uma **data fixa** (15/05/2000) e exibir o **dia, mês e ano** separadamente.

Use os métodos do objeto `Date`.

 **Dica:** Use `getDate()`, `getMonth() + 1` e `getFullYear()`.

atividade-3.js

```
// ATIVIDADE 3: Acessando partes da data fixa
// Complete as lacunas

let data = new Date(2000, 4, 15);

console.log("Dia:" + data.[ ? ]());
console.log("Mês:" + (data.[ ? ]() + 1));
console.log("Ano:" + data.[ ? ]());
```

 Cole aqui o **código COMPLETO** que você escreveu (com as lacunas preenchidas):

```
let data = new Date(2000, 4, 15);

console.log("Dia:" + data.getDate());
console.log("Mês:" + (data.getMonth() + 1));
console.log("Ano:" + data.getFullYear());
```

 Verificar

 Limpar

 **Perfeito! Código completo e correto!**

 Todas as lacunas foram preenchidas corretamente!

 **Código esperado (completo):**

```
let data = new Date(2000, 4, 15);
console.log("Dia:" + data.getDate());
console.log("Mês:" + (data.getMonth() + 1));
console.log("Ano:" + data.getFullYear());
```

4 Calculando a Idade

Enunciado:

Complete a função `calcularIdade` para calcular a idade a partir da data de nascimento.

Use a data fixa **15/05/2000** para testar.

 **Dica:** Subtraia o ano de nascimento do ano atual e verifique se a pessoa já fez aniversário.

atividade-4.js

```
// ATIVIDADE 4: Calculando a idade
// Complete as lacunas

function calcularIdade(dataNascimento) {
    let hoje = new Date();
    let nascimento = new Date(dataNascimento);
    let idade = hoje.[ ? ]() - nascimento.[ ? ]();

    // Verifica se já fez aniversário este ano
    if (hoje.[ ? ]() < nascimento.[ ? ]()) {
        idade--;
    }

    return idade;
}

// Testando com data fixa
let idade = calcularIdade("2000-05-15");
console.log("Idade:" + idade);
```

 Cole aqui o **código COMPLETO** que você escreveu (com as lacunas preenchidas):

```
    return idade;
}

let idade = calcularIdade("2000-05-15");
console.log("Idade:" + idade);
```

 Verificar

 Limpar

 **Perfeito! Código completo e correto!**

✓ Todas as lacunas foram preenchidas corretamente!



Código esperado (completo):

```
let idade = hoje.getFullYear() - nascimento.getFullYear();
if (hoje.getMonth() < nascimento.getMonth()) {
    idade--;
}
return idade;
}
let idade = calcularIdade("2000-05-15");
console.log("Idade:" + idade);
```

5 Função Callback

📌 Enunciado:

Complete o código para criar uma função que recebe uma **callback** e a executa.

A callback deve exibir uma mensagem no console.

💡 **Dica:** Callback é uma função passada como argumento. Ela é executada dentro da função principal.

atividade-5.js

```
// ATIVIDADE 5: Função Callback
// Complete as lacunas

function executar([ ? ]) {
    console.log("Executando callback...");
    [ ? ]();
}

executar(function() {
    console.log("Olá, mundo!");
});
```

📋 Cole aqui o **código COMPLETO** que você escreveu (com as lacunas preenchidas):

```
    console.log("Executando callback...");
    callback();
}

executar(function() {
    console.log("Olá, mundo!");
});
```

✅ Verificar

🧼 Limpar

🎉 **Perfeito! Código completo e correto!**

✅ Todas as lacunas foram preenchidas corretamente!

💡 **Código esperado (completo):**

```
function executar(callback) {
    console.log("Executando callback...");
    callback();
}

executar(function() {
    console.log("Olá, mundo!");
});
```

```
});
```


6 Callback com Parâmetros

📌 Enunciado:

Complete a função `operarNumeros` que recebe **dois números** e uma **callback**.

A função deve executar a callback passando os dois números.

💡 **Dica:** A callback recebe os números como parâmetros e retorna o resultado.

atividade-6.js

```
// ATIVIDADE 6: Callback com Parâmetros
// Complete as lacunas

function operarNumeros(a, b, [ ? ]) {
    let resultado = [ ? ](a, b);
    console.log("Resultado:", resultado);
}

operarNumeros(5, 3, function(x, y) {
    return x + y;
});
```

📋 Cole aqui o **código COMPLETO** que você escreveu (com as lacunas preenchidas):

```
function operarNumeros(a, b, callback) {
    let resultado = callback(a, b);
    console.log("Resultado:", resultado);
}

operarNumeros(5, 3, function(x, y) {
    return x + y;
});
```

✅ Verificar

🗑 Limpar

🎉 **Perfeito! Código completo e correto!**

✅ Todas as lacunas foram preenchidas corretamente!

💡 **Código esperado (completo):**

```
function operarNumeros(a, b, callback) {
    let resultado = callback(a, b);
    console.log("Resultado:", resultado);
}

operarNumeros(5, 3, function(x, y) {
    return x + y;
});
```

}):

7 CRUD com Array

Enunciado:

Complete as operações CRUD (Create, Read, Update, Delete) com um array de tarefas.

Use os métodos de array aprendidos anteriormente.

 **Dica:** `push()` para adicionar, `splice()` para remover, e atribuição direta para atualizar.

atividade-7.js

```
// ATIVIDADE 7: CRUD com Array
// Complete as operações

let tarefas = [];

// CREATE - Adicionar
tarefas[ ? ]("Estudar JavaScript");
tarefas[ ? ]("Fazer exercícios");

// READ - Consultar
console.log("Tarefas:", tarefas);

// UPDATE - Atualizar (primeira tarefa)
tarefas[0] = "Estudar JavaScript Avançado";

// DELETE - Remover (primeira tarefa)
tarefas[ ? ](0, 1);

console.log("Tarefas atualizadas:", tarefas);
```

 Cole aqui o **código COMPLETO** que você escreveu (com as lacunas preenchidas):

```
tarefas[0] = "Estudar JavaScript Avançado";

tarefas.splice(0, 1);

console.log("Tarefas atualizadas:", tarefas);
```

 Verificar

 Limpar



Perfeito! Código completo e correto!

✓ Todas as lacunas foram preenchidas corretamente!

💡 **Código esperado (completo):**

```
        let tarefas = [];  
tarefas.push("Estudar JavaScript");  
tarefas.push("Fazer exercícios");  
console.log("Tarefas:", tarefas);  
tarefas[0] = "Estudar JavaScript Avançado";  
tarefas.splice(0, 1);  
console.log("Tarefas atualizadas:", tarefas);
```

8 Convertendo CSV para Array

📌 Enunciado:

Complete o código para converter um texto CSV em um array de objetos.

Use `split()` para separar linhas e colunas.

💡 **Dica:** Primeiro divida em linhas (`\n`), depois divida cada linha em colunas (`,`).

atividade-8.js

```
// ATIVIDADE 8: Convertendo CSV
// Complete as lacunas

let csv = "nome,idade\nAna,25\nJoão,30";

// Dividir em linhas
let linhas = csv.[ ? ]("\n");

// Dividir a primeira linha em cabeçalhos
let cabecalhos = linhas[0].[ ? ](",");

// Percorrer as linhas de dados
for (let i = 1; i < linhas.length; i++) {
    let valores = linhas[i].[ ? ](",");
    console.log(cabecalhos[0] + ":" + valores[0]);
}
```

📋 Cole aqui o **código COMPLETO** que você escreveu (com as lacunas preenchidas):

```
let cabecalhos = linhas[0].split(",");

for (let i = 1; i < linhas.length; i++) {
    let valores = linhas[i].split(",");
    console.log(cabecalhos[0] + ":" + valores[0]);
}
```

✅ Verificar

🗑 Limpar

🎉 **Perfeito! Código completo e correto!**

✅ Todas as lacunas foram preenchidas corretamente!

💡 **Código esperado (completo):**

```
let csv = "nome,idade\nAna,25\nJoão,30";
```

```
let linhas = csv.split("\n");
let cabecalhos = linhas[0].split(",");
for (let i = 1; i < linhas.length; i++) {
  let valores = linhas[i].split(",");
  console.log(cabecalhos[0] + ":" + valores[0]);
}
```



Resultado Final

8 / 8

- ✓ Atividade 1 - Correta
- ✓ Atividade 2 - Correta
- ✓ Atividade 3 - Correta
- ✓ Atividade 4 - Correta
- ✓ Atividade 5 - Correta
- ✓ Atividade 6 - Correta
- ✓ Atividade 7 - Correta
- ✓ Atividade 8 - Correta